

The Verified Rulebook

One-page overview of Noesis’s two-layer verification design — the composite of the UTXO rulebook, the recursive zk validity proof, formal verification, and the Pragma amendment-coherence layer. It ties together the stateless-verification phase docs (docs/rulebook-map.md , docs/phase1-extraction-report.md , docs/phase2-commitment-report.md , docs/phase3-zk-plan.md , docs/phase4-fv-plan.md) and the amendment socket (docs/DESIGN-pragma-layer-amendment-coherence.md).

Status discipline:  built ·  designed / partly-built ·  designed-only. Never round up. **Anti-hallucination:** every file:line is a pointer to re-verify at source before relying.

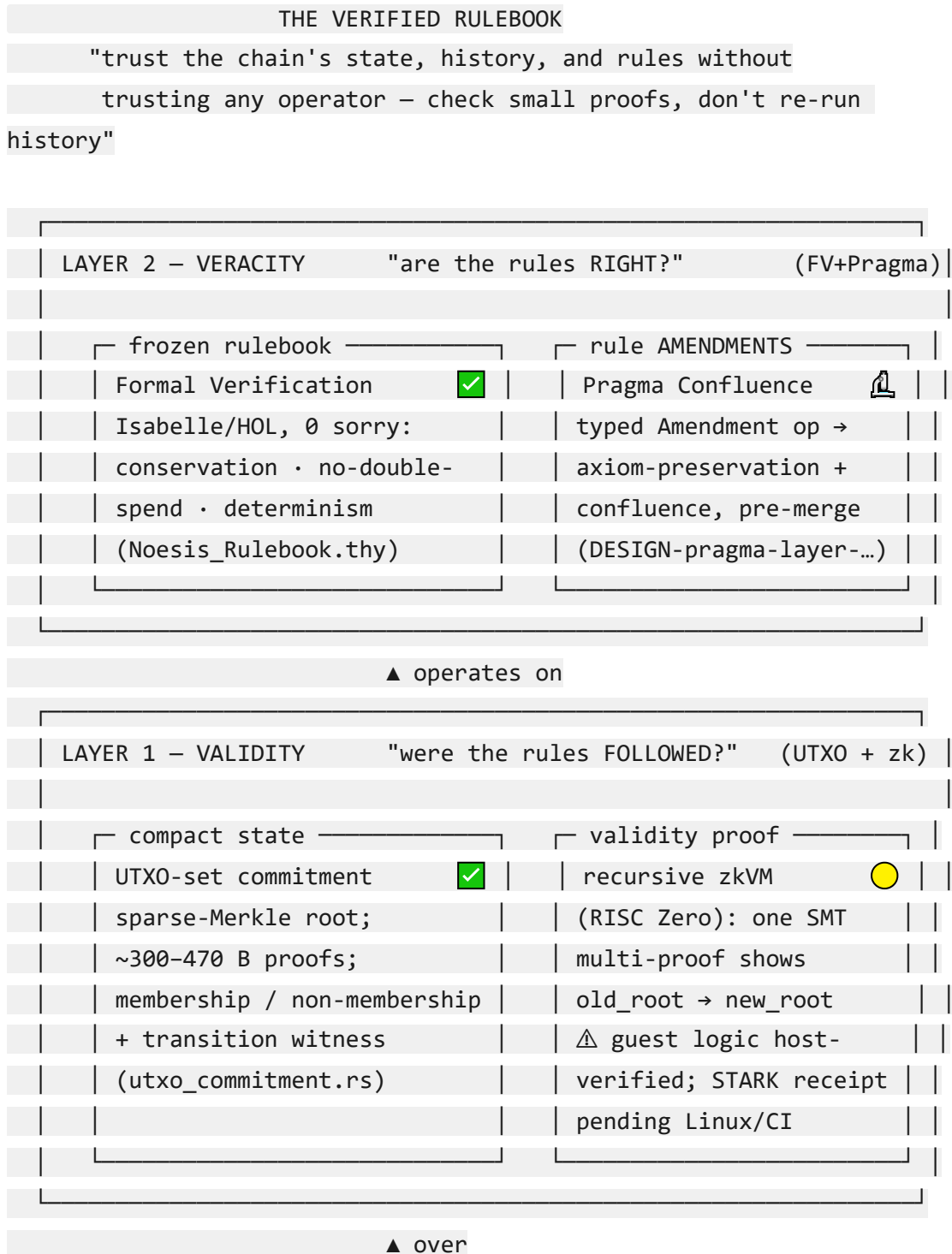
The claim

A node can trust the chain’s state, history, and rules without trusting any operator — by checking small proofs, not by re-running history. It does this on two layers:

Layer	Question	Your grouping	Built from
Layer 1 — Validity	were the rules followed?	UTXO rulesets + zk recursion	compact UTXO-set commitment + a recursive zkVM validity proof
Layer 2 — Veracity	are the rules right?	FV + Pragma integration	formal verification of the frozen rulebook + Pragma Confluence over amendments

Both rest on a **STATE-able rulebook**: the pure, extractable transition function `runtime::apply_block` (Phase 0/1). Validity proves *this execution* obeyed the rulebook; Veracity proves *the rulebook itself* is sound — Layer 1 for the fixed rulebook, plus a second line of defence for the space of governable rulebooks.

Structure



BASE – the RULEBOOK, made STATE-able 

runtime::apply_block(state, block, params) -> Ledger | Violation |

pure · deterministic · integer · extractable (Phase 0 / 1)

OUT OF SCOPE – stays with consensus (NCI pow/pos/pom):

X canonicity (which fork) X data availability (a root ≠ the data)

A node's experience

A NODE VALIDATES THE CHAIN – what it actually does

— WITHOUT the stack (today's default) ————— cost:
O(chain)

download full history → re-execute every block from → trust = "I re-ran

(hours, needs full state) genesis myself
everything"

— WITH the Verified Rulebook ————— cost:
O(1)-ish

① fetch: latest UTXO-commitment root + one recursive proof

② VALIDITY – verify the zk receipt against that root 

⇒ "every block since genesis obeyed apply_block"

(checks a ~KB proof; NEVER re-executes a block)

③ VERACITY – lean on the machine-checked proof that apply_block

| itself conserves value & forbids double-spend 

|

| ⇒ "the rules it followed are RIGHT, not just self-consistent"|

|

|

| ④ read/sync any coin via a ~300-470 B membership proof 

|

⇒ node trusts state + history with ZERO operator trust, holding no full copy and re-running nothing.

— WHEN GOVERNANCE CHANGES THE RULES —

amendment → Pragma coherence gate (pre-merge, sub-second) 

(typed op) | confluent? — axioms kept? — anonymity-relax kept?

| all yes → amendment merges

| any no → amendment REJECTED

⇒ the node never has to trust that a rule-change stayed sound; the gate proved it BEFORE the new rule took effect.

Still supplied by consensus, NOT by this stack:

- which fork is canonical
- the block data itself

Honest status ledger

Component	Axis	Status	Where (re-verify)
Extractable pure rulebook apply_block	base (STATE)		node/src/runtime.rs (Phase 0/1); replay-parity node/tests/apply_block_parity.rs
UTXO-set commitment (sparse-Merkle)	Layer 1		node/src/utxo_commitment.rs; vendored pure-Rust SMT onchain/vendor/sparse-merkle-tree (Phase 2). ~300-470 B proofs
Transition witness	Layer 1		utxo_commitment::{transition, verify_transition,

Component	Axis	Status	Where (re-verify)
(old_root → new_root)			TransitionWitness} — host-verified
Recursive zkVM validity proof	Layer 1		guest logic host-verified GREEN; real STARK receipt pending a Linux/WSL2/CI prover (RISC Zero). docs/phase3-zk-plan.md , onchain/zk-utxo/ internal/fv/Noesis_Rulebook.thy
FV of the frozen rulebook	Layer 2		(conservation + no_double_spend + determinism, Isabelle2025, 0 sorry); node/tests/fv_invariants.rs ; reproducible via internal/fv/verify.sh
Pragma amendment coherence	Layer 2		socket designed (docs/DESIGN-pragma-layer-amendment-coherence.md); Constitution amendment rules still pending in node/src/runtime.rs ; integration terms-first

Do not claim “trustless full verification” without naming the two boundaries below — a receipt proves the rulebook was followed and FV proves it is right, but neither settles *which fork* is canonical or whether *the data behind a root* is available.

Boundaries (load-bearing)

- **Canonicity** (which fork wins) stays with consensus — the NCI mix (PoW / PoS / PoM), PoW excluded from finality, anti-concentration floor. The Verified Rulebook says *a* chain is valid-and-sound; it does not choose *the* chain.
- **Data availability** — a commitment root is not the data. Availability stays with consensus / the network.

Why two layers, not one

A validity proof (Layer 1) can prove a block perfectly obeyed a rulebook that is itself wrong — a proof that the chain conserved value under a rule that permits inflation is worthless. FV (Layer 2) closes the *“is the rule right”* gap a zk receipt cannot. And FV alone proves a *fixed* rulebook; when governance amends the rulebook, Pragma Confluence extends the guarantee to the space of reachable rulebooks (the dangerous quadrant — *Confluent + Axiom-breaking* — see the design note). Layer 1 defends each execution; Layer 2 defends the rules, frozen and evolving.

Companion reading: [docs/phase4-fv-plan.md](#), [docs/DESIGN-pragma-Layer-amendment-coherence.md](#), [docs/Pragma OverLaps/noesis-pragma-overLap.md](#), and the phase reports ([docs/rulebook-map.md](#) → [docs/phase2-commitment-report.md](#) → [docs/phase3-zk-plan.md](#)).